Do Developers Use SAST Tools Straight Out of the Box?

> A 2024 large-scale survey of 1,263 developers showing that SAST tools are rarely combined or configured — and that running them on default settings leaves significant vulnerabilities undetected.

Metadata

- **Author(s):** Gareth Bennett, Tracy Hall, Emily Winter, Steve Counsell, Thomas Shippey
- **Publication:** Proceedings of the 18th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM '24), Barcelona, Spain
- **Date:** 2024-10-24
- **Reading Time:** ~20 minutes (7-page paper)
- **Level:** Intermediate
- **Link:** Read the full paper (ACM, open access) · Open PDF (Brunel/Lancaster repository)
- **License:** Creative Commons Attribution 4.0 (open access)

Summary

SAST tools are an established way to detect vulnerabilities early, but prior work shows they have low detection rates unless teams either **combine multiple tools** or **configure their rule sets**. Crucially, earlier research found that *configuring a single tool's rule set has been shown to outperform combining several tools used out of the box*. This paper asks a deceptively simple question: do developers actually do either of these things, or do they just run SAST on its defaults?

To answer it, the authors surveyed **1,263 developers** (recruited via Prolific, LinkedIn, and developer communities, with pre-screening to filter out respondents who claimed to use fake tools). After quality filtering, **1,003 valid responses** remained.

The results are sobering. Only **20% (204/1,003) used SAST tools at all.** Among those who did, a large share never moved beyond defaults: **59% did not use multiple tools, 54% did not configure their tools, and 40% did neither.** In other words, the majority of SAST users run a single tool, unconfigured — precisely the setup prior evidence shows is least effective.

The authors conclude that more work is needed to help developers combine and configure tools, because doing so is likely to detect significantly more vulnerabilities. They also surface a disconnect between what tool providers ship as defaults and what users actually need.

Key Takeaways

1. **SAST adoption is still low.** Only ~1 in 5 surveyed developers used any SAST tool, more than a decade after these tools became mainstream.
2. **Defaults dominate.** A majority of SAST users neither configure rule sets nor combine tools — leaving the tool in its weakest, noisiest configuration.

3. **Out-of-the-box ≠ effective.** Configuring a single tool's rules can outperform stacking several tools on defaults, yet configuration is exactly what developers skip.
4. **A provider/user gap exists.** Why and how developers configure tools points to a mismatch between vendor defaults and real-world needs.

Why This Matters

This is the modern, evidence-backed update to the long-standing observation that SAST tools are underused. It moves the conversation past "do developers use SAST?" to the sharper, more actionable question: **"are they using it well?"** — and the answer is largely no.

For a DevSecOps pipeline, the implication is direct and practical: **adopting a SAST tool is the easy 20%; the value is in configuration.** A scanner left on defaults produces the high false-positive, low-precision experience that erodes trust and drives abandonment. The practices that fix this — curated/tuned rule sets, baselining to focus on new code, severity thresholds, and combining a fast pattern engine with a deeper data-flow engine — are not optional polish. They are the difference between a tool that finds real bugs and one that ships noise.

Practical Applications

- **Treat configuration as part of adoption, not an afterthought.** Ship a tuned, team-approved rule set with the tool from day one rather than the vendor default.
- **Combine complementary engines deliberately.** Pair a fast pattern scanner (PR stage) with a deeper data-flow engine (scheduled stage) instead of relying on a single default tool.
- **Baseline and threshold.** Focus blocking on new, high-severity findings so default-induced noise doesn't bury real issues.
- **Review your defaults periodically.** The provider/user gap the paper highlights means vendor defaults rarely match your stack's risk profile.

Notable Quotes

> "Of those developers who did use SAST tools, a large number did not use multiple tools (59%), did not configure tools (54%) or did neither (40%)."

> "Our results suggest that more work is needed to help developers combine and configure tools, since doing so is likely to detect significantly more vulnerabilities."

Related Topics

- SAST rule-set tuning and configuration
- Combining complementary SAST engines (pattern + data-flow)
- False positives, baselining, and developer trust
- "Shift-left" security and developer experience

Further Reading

- An Empirical Study of Static Analysis Tools for Secure Code Review (arXiv, 2024) — evaluates CodeQL, CodeChecker, Infer, Flawfinder, and Cppcheck on 815 real vulnerable commits.

- Barriers to Using SAST Tools: A Literature Review (ASEW 2024)
- OWASP Source Code Analysis Tools
- Static Analysis (SAST) Blueprint — applies these findings as a concrete, tuned pipeline strategy.

Critical Analysis

Strengths: A large, recent (2024) sample with explicit quality screening, which lends weight to its headline statistics. It quantifies a problem the field had mostly described anecdotally, and it isolates a specific, fixable lever — *configuration* — rather than vague "adoption."

Limitations / how it ages: It is a self-reported survey, so actual configuration behavior may differ from what respondents claim, and the Prolific-heavy recruitment skews toward a particular developer population. It is described by the authors as an emerging-results paper, so the findings are directional. It also predates the rapid rise of LLM-assisted triage and auto-configuration, which may shift the "defaults vs. tuned" calculus in coming years.

Tags

`sast` `static-analysis` `configuration` `tool-adoption` `false-positives` `empirical-study` `devsecops`

Contributed by: @ruimachado23
Last Updated: 2026-06-10
Relevance: High — recent, quantitative evidence that SAST value comes from configuration and tool combination, directly underpinning this chapter's "tune for signal" blueprint.